Medixmart Plus: Project Blueprint and 8-Week Work Plan

Start date: October 5, 2026
Handover date: November 29, 2026
Method: Agile, four 2-week sprints
Team: 6 members (roles are attached to member numbers, so names can be filled in later)

---

1. What Are We Building? (The Simple Story)

Imagine a small pharmacy. Today the owner writes things in notebooks: which medicines are on the shelf, when they go bad, who owes money. Notebooks get lost, and nobody can see problems early.

Medixmart Plus is a computer and phone app that does all this for them:

The pharmacy needs to...	The app will...
Know what medicines it has	Keep a list of every medicine and how many are left
Not sell expired medicine	Warn when a medicine will expire in 7 months or less
Know when to re-order	Warn when a medicine is running low
Sell things quickly	Have a checkout screen (POS = Point of Sale) that makes a bill
Sell the oldest stock first	Automatically take from the batch that expires soonest (called **FIFO**)
Remember customers who pay later	Keep a "credit" record of who owes what
Be friendly to customers	Send birthday messages
Return old medicine to suppliers	Show a list of medicines close to expiry
Understand their business	Make sales and stock reports they can open in Excel

---

2. The Big Picture: How the Pieces Fit Together

Think of the app like a restaurant:

- **Flutter app = the dining room.** What the staff see and tap (screens, buttons).

- **Node.js/Express server = the waiter and kitchen.** It takes requests from the screen, does the work, and brings back an answer.
- **PostgreSQL database = the pantry.** Where all the information is stored safely.



Technologies and why we use them

Technology	What it is (simple words)	What we use it for
Flutter (Dart language)	A toolkit that builds one app that runs on both Windows and Android	All screens
Provider	A way for screens to share information (like the cart)	State management in Flutter
Node.js + Express	JavaScript that runs on a computer to make a server	The backend and APIs
PostgreSQL	A very organised digital filing cabinet	Database
JWT (JSON Web Token)	A digital "wristband" given after login so the app remembers you are allowed in	Login security
bcrypt	A scrambler that hides passwords so nobody can read them	Password safety
dotenv (.env file)	A private notes file for secrets (passwords, keys)	Keeping secrets out of the code
node-cron	An alarm clock for the server	Daily expiry check
Firebase Cloud Messaging (FCM)	Google's free message-delivery service	Birthday notifications
Figma	A drawing board for screen designs	UI/UX design
GitHub	A shared online folder that remembers every change	Teamwork on code
Postman	A tool to test the server without needing screens	API testing
Flutter `pdf` + `printing` **packages**	Make PDF files inside the app	Receipts and invoices
`csv` / `excel` **packages**	Make spreadsheet files	Exportable reports

> **Words you will see: API** = a doorway on the server the app knocks on (example: `/api/medicines`). **Endpoint** = one specific doorway. **JSON** = the simple text format used to send data. **CRUD** = Create, Read, Update, Delete. **PR** = Pull Request, asking teammates to

> check your code before it joins the main project. **UAT** = User Acceptance Testing, where real users try the app and say "yes, this works for us."

---

3. The Team (Roles by Member Number)

Member	Role	In one sentence
Member 1	System Architect and Backend Lead	Builds the server's skeleton, security, and daily jobs. Checks everyone's backend code.
Member 2	Frontend and UI/UX Lead	Designs and builds the look of the app.
Member 3	Inventory and Batch Management	Medicines, suppliers, batches, expiry, alerts.
Member 4	Sales and Billing Engine	The checkout, stock deduction, and receipts.
Member 5	Customer Credit and Messaging	Customers, who owes money, birthday messages, staff training.
Member 6	Analytics, QA and Documentation	Tests everything, writes reports and project papers, builds the exports.

If roles change, just swap the member numbers in this document. The tasks stay the same.

---

4. Rules for Working Together (Do This Every Week)

1. **Daily 10-minute stand-up** (chat or call). Each person answers: What did I do? What will I do today? Am I stuck?
2. **Never work directly on the `main` branch.** Make your own branch (example: `feature/login-api`), then open a Pull Request.
3. **Every PR needs at least 1 reviewer** (Member 1 reviews backend, Member 2 reviews Flutter code).
4. **Test before you say "done."** A task is only done when it works, has been tested in Postman or on screen, and has been merged.
5. **Sprint planning on day 1 of each sprint** (Mondays Oct 5, Oct 19, Nov 2, Nov 16) and **a review and retrospective on the last day** (Sundays Oct 18, Nov 1, Nov 15, Nov 29). In the retro, talk about what went well, what went badly, and what to change.

6. **Use a task board** (GitHub Projects or Trello) with columns: To Do, Doing, In Review, Done.
7. **Ask for help early.** Stuck for more than 1 hour? Tell the team.

Standard folder layout (so everyone can find things)

```
medixmart-backend/
  src/
    config/        (database connection, settings)
    routes/        (list of doorways: auth.js, medicines.js ...)
    controllers/   (the work done behind each doorway)
    middleware/    (checks that run before the work: login check, error handler)
    jobs/          (node-cron daily jobs)
    sql/           (database scripts)
  .env             (secrets, never uploaded to GitHub)
  server.js

medixmart_app/ (Flutter)
  lib/
    screens/       (login, dashboard, pos, medicines ...)
    providers/     (shared state: cart, auth ...)
    services/      (code that talks to the server)
    models/        (shapes of data: Medicine, Batch ...)
    widgets/       (small reusable pieces)
```

---

5. The Database Plan (The Pantry)

Think of each **table** as a spreadsheet. Here are the main ones:

Table	What it holds	Important columns
`users`	Staff who can log in	id, name, username, password_hash, role
`categories`	Medicine groups (Tablets, Syrups...)	id, name
`medicines`	The medicine itself	id, name, category_id, unit_price, reorder_level
`suppliers`	Companies we buy from	id, name, phone, address
`batches`	A delivery of one medicine from one supplier	id, medicine_id, supplier_id, batch_no, quantity, cost_price, **expiry_date**, received_date
`customers`	Pharmacy customers	id, name, phone, birthday, credit_balance
`sales`	One bill	id, date, customer_id, total, payment_type (cash/credit)
`sale_items`	Each line on a bill	id, sale_id, medicine_id, **batch_id**, quantity, price
`credit_payments`	When a customer pays back	id, customer_id, amount, date
`alerts`	Warnings	id, type (expiry/low_stock), medicine_id, message, is_read, created_at

Why "batch" matters: A pharmacy may have 100 Paracetamol that expire in March and another 100 that expire in September. They are the same medicine but different batches. We must track them separately so we sell the March ones first.

Relationships (simple): one Medicine has many Batches. One Supplier has many Batches. One Sale has many Sale Items. Each Sale Item points to the exact Batch it was taken from.

---

6. Sprint-by-Sprint Plan

SPRINT 1: Foundation and Architecture (Oct 5 to Oct 18)

Goal in simple words: Build the "empty house." Everyone sets up their tools, designs are drawn, and the app can log in and talk to the server.

Sprint 1 is successful when: a user can type a username and password in the Flutter app, the server checks it, and the app opens an empty dashboard.

Week 1 (Oct 5 to Oct 11): Setup and Design

Member 1 (Backend Lead): Start the server

What: Create the Node.js/Express project and connect it to PostgreSQL.
How:

1. Install Node.js (LTS version) and PostgreSQL. Install pgAdmin (a friendly viewer for the database).
2. In a new folder run `npm init -y`, then `npm install express pg dotenv cors bcrypt jsonwebtoken node-cron`.
3. Create a `.env` file with `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `PORT`.
4. Write `src/config/db.js` that creates a connection pool using those values.
5. Create `server.js` with a test route: `GET /api/health` that replies `{ "status": "ok" }`.
6. Add `.env` and `node_modules` to `.gitignore`.

Done when: opening `http://localhost:3000/api/health` shows "ok" and the server can run a simple query like `SELECT NOW()`.

Member 2 (Frontend Lead): Design the screens

What: Draw how the app will look before anyone builds it.
How:

1. Create a free Figma account and a team workspace. Invite all members.
2. Set a **design system** first: 2 or 3 brand colours (pharmacy green/blue works well), one font, and button/input styles. This keeps all screens consistent.
3. Design three high-fidelity screens: **Login**, **Dashboard** (cards for Total Sales, Low Stock, Expiring Soon), and **POS** (search bar on top, cart list, total, Pay button).
4. Make a desktop version (wide) and a mobile version (narrow) of each.
5. Share the Figma link and ask the team for feedback.

Done when: the team has approved the three screens.

Member 3 (Inventory): Create the tables

What: Turn the database plan (Section 5) into real tables.
How:

1. Write a file `sql/01_create_tables.sql` containing `CREATE TABLE` commands for medicines, categories, suppliers, batches (and the rest).
2. Use `PRIMARY KEY` for IDs and `FOREIGN KEY` to link tables (example: `batches.medicine_id` points to `medicines.id`).

3. Use `NOT NULL` for things that must exist (like expiry_date) and `CHECK (quantity >= 0)` so stock can never go negative.
4. Add indexes on `batches(expiry_date)` and `batches(medicine_id)` so searches are fast.
5. Run the script in pgAdmin. Save it in GitHub so everyone can recreate the same database.

Done when: all tables exist and each team member can run the script on their own computer.

Member 4 (Sales): Plan the checkout "conversation"

What: Decide exactly what the app sends to the server when a sale is made, and what it gets back.
How:

1. Write the **request** (what the app sends), for example:

```json
{
  "customer_id": 12,
  "payment_type": "cash",
  "items": [
    { "medicine_id": 5, "quantity": 3 },
    { "medicine_id": 9, "quantity": 1 }
  ]
}
```

2. Write the **response** (what the server answers), for example:

```json
{
  "sale_id": 101,
  "total": 1450.00,
  "items": [
    { "medicine": "Paracetamol", "quantity": 3, "batches_used": [{ "batch_id": 4,
  ]
}
```

3. Also write the **error replies** (not enough stock, medicine not found).
4. Put this in a shared document and review it with Members 1 and 2.

Done when: the JSON format is agreed by everyone.

Member 5 (Customers and Messaging): Set up Firebase

What: Create the "message sender" account.
How:

1. Go to the Firebase Console and create a project called `medixmart-plus`.
2. Add an Android app (for the phone) and a Web/Windows config if needed.
3. Download `google-services.json` and generate a **service account key** for the server (this lets the Node.js server send messages).
4. Store keys safely. **Never put them on GitHub.** Share them privately with Member 1.
5. Make a short note on how to send a test message.

Done when: a test notification can be sent from the Firebase console.

Member 6 (QA and Docs): Set up GitHub and Postman

What: Create the shared home for the code and the testing tool.
How:

1. Create the GitHub repository (or two: backend and app). Add a `README.md`.
2. Add everyone as collaborators.
3. Under **Settings > Branches**, add a rule for `main`: require a Pull Request and at least 1 approval.
4. Create branches `main` and `develop`.
5. Create a **Postman team workspace**, a collection called "Medixmart API," and an **environment** with variables like `{{base_url}}` and `{{token}}`.

Done when: nobody can push straight to `main` and everyone can open the Postman workspace.

Week 2 (Oct 12 to Oct 18): Login and Navigation

Member 1: Build login

What: `POST /api/auth/login`
How:

1. When creating a user, hash the password with `bcrypt.hash(password, 10)` before saving.
2. On login: find the user by username, then use `bcrypt.compare(...)`. If it matches, create a token with `jwt.sign({ id, role }, JWT_SECRET, { expiresIn: '8h' })` and send it back.
3. Write a middleware `authCheck` that reads the token from the `Authorization: Bearer ...` header and blocks the request if it is missing or wrong. Every other API will use it.

4. Add a seed script that creates one admin user.

Done when: correct password gives a token; wrong password gives a 401 error.

Member 2: Start Flutter

What: Create the app and the Login screen.
How:

1. Run `flutter create medixmart_app`. Enable Windows desktop: `flutter config --enable-windows-desktop`.
2. Add packages in `pubspec.yaml`: `provider`, `http` (or `dio`), `go_router`, `flutter_secure_storage`.
3. Set up navigation routes: `/login`, `/dashboard`, `/pos`, `/medicines`, and so on.
4. Create an `AuthProvider` that holds the token and says whether the user is logged in.
5. Build the Login screen from the Figma design (two text boxes, a button, an error message).
6. Create `api_service.dart`, one place that holds the server address and adds the token to every request.

Done when: the Login screen appears and tapping Login calls Member 1's API.

Member 3: Fill the database with fake data

What: Test data so everyone can build screens without typing medicines by hand.
How:

1. Write `sql/02_dummy_data.sql` with about 5 categories, 5 suppliers, 30 medicines, and 60 batches.
2. Make the batches varied on purpose: some expiring in 2 months, some in 6, some in 2 years; some with quantity 2 (low stock). This helps test alerts later.
3. Add a few test customers (one with a birthday near the test date, one with credit owed).

Done when: `SELECT * FROM medicines;` shows realistic rows.

Member 4: POS screen shell

What: The POS screen looks right but is not connected yet.
How:

1. Build the layout: search bar on top, a list of search results, a cart panel on the side (empty for now), and a total area.
2. Create a `CartProvider` with an empty list (and placeholder add/remove methods).
3. Use fake local data to show how items would look in the cart.

Done when: the POS screen matches Figma.

Member 5: Customer APIs

What: `POST /api/customers`, `GET /api/customers`, `GET /api/customers/:id`, `PUT /api/customers/:id`.
How:

1. Use Express routes + controller functions with SQL queries (`INSERT`, `SELECT`, `UPDATE`).
2. Validate input: name and phone are required; phone must look like a phone number.
3. Protect all routes with Member 1's `authCheck`.
4. Return clear errors (400 for bad data, 404 if not found).

Done when: all four calls work in Postman.

Member 6: First tests and first document

What: Postman tests and Chapter 1 of the academic document (SRS = Software Requirements Specification).
How:

1. In Postman, write tests for login and customers. Example test: "status is 200" and "response has a token." Save token into `{{token}}` automatically.
2. Test the bad cases too: wrong password, missing fields.
3. Start SRS Chapter 1: introduction, purpose, scope, users of the system, problem statement, objectives.

Sprint 1 review (Oct 18): Demo login to the team. Run the retrospective.

SPRINT 2: Core Inventory and Batch Management (Oct 19 to Nov 1)

Goal in simple words: Staff can add medicines, record deliveries from suppliers (batches), and see what is in stock.

Sprint 2 is successful when: you can add a new medicine, receive a batch of it from a supplier with an expiry date, and see the updated stock in the app.

Week 3 (Oct 19 to Oct 25): Medicine CRUD

Member 1: Safety net for errors

What: Error-handling middleware and code reviews.
How:

1. Write one function at the end of `server.js` that catches any error and replies in a standard shape: `{ "success": false, "message": "..." }`.
2. Create a small helper to wrap async routes so errors are caught automatically.
3. Review all open Pull Requests daily. Check: does it work, is it readable, is there no secret in the code?

Member 2: Medicine screens

What: "Medicine List" and "Add/Edit Medicine" forms.
How:

1. List screen: a searchable table (desktop) or list (mobile) showing name, category, price, total stock.
2. Form screen: text boxes for name and price, a dropdown for category, a reorder level field, Save button.
3. Use `Form` and `TextFormField` with validators (name cannot be empty, price must be a number).
4. Create a `MedicineProvider` that loads the list from the API.

Member 3: Medicine and Category APIs

What: Endpoints for medicines and categories.
How:

1. `GET/POST /api/categories`.
2. `GET /api/medicines` (with search `?q=para` and total stock calculated by adding up the batch quantities), `GET /api/medicines/:id`, `POST`, `PUT`, and a safe "deactivate" instead of a hard delete (so old sales do not break).
3. Check duplicates (same medicine name twice).

Member 4: "Receive Stock" form

What: The form staff use when a delivery arrives.
How:

1. Fields: Supplier (dropdown), Medicine (searchable dropdown), Batch number, Quantity, Cost price, Expiry date (date picker), Received date.
2. Validate that expiry date is in the future and quantity is more than 0.
3. For now, show a "Saved" message without calling the server (the API is built next week).

Member 5: Customer Directory screen

What: A list of customers.
How:

1. Build a searchable list showing name, phone, and credit balance (highlight in red when more than 0).
2. Add a screen to add or edit a customer, using the APIs from Sprint 1.
3. Create a `CustomerProvider`.

Member 6: Testing and System Design document

What: Test the medicine APIs and write the design section.
How:

1. Postman tests for add, edit, search, and duplicate medicine.
2. Draw the **ER diagram** (tables and links, like Section 5), a **Use Case diagram** (who does what: Admin, Pharmacist), and a simple **architecture diagram** (Section 2). Use draw.io or PlantUML.
3. Write the "System Design" section explaining each diagram in plain words.

Week 4 (Oct 26 to Nov 1): Batch Tracking

Member 1: Help connect screens to the server

How:

1. Make sure `cors()` is enabled in Express so the app is allowed to call the server.
2. Help teammates debug API calls (wrong URL, missing token, wrong JSON).
3. Write a short "How to call our API from Flutter" guide with a working example.

Member 2: Supplier screens

How:

1. Supplier list and Add/Edit supplier forms (name, phone, address).
2. Connect to the supplier API (Member 3 may need to build `GET/POST/PUT /api/suppliers` first. Agree this on Monday).
3. Reuse the form widgets from the medicine screens.

Member 3: Batch API

What: `POST /api/batches` and `GET /api/batches?medicine_id=...`
How:

1. Check that the medicine and supplier exist.
2. Check the expiry date is in the future and the quantity is above 0.
3. Insert into `batches`.
4. `GET` returns batches sorted by `expiry_date` ascending (soonest first). This sorting is the heart of FIFO later.

Member 4: Connect Receive Stock to the API

How:

1. Create `batch_service.dart` that sends the form to `POST /api/batches`.
2. Show a loading spinner while waiting; show a success or error message.
3. After success, clear the form and refresh the medicine list.

Member 5: Credit flag logic

What: Backend logic to mark customers with unpaid balances.
How:

1. Add to `GET /api/customers` a field `has_outstanding_credit` (true when `credit_balance > 0`).
2. Add `GET /api/customers/credit` which lists only customers who owe money, biggest first.
3. Create `POST /api/customers/:id/payments` so a payment lowers the balance and is stored in `credit_payments`.

Member 6: Integration testing

What: Check that the pieces work together.
How:

1. Test: add a batch in the app, then look in the database. Is the row there with the right values?
2. Test: add a bad batch (past expiry date). It must be rejected.
3. Write a **test case table** (ID, steps, expected result, actual result, pass/fail) and keep it growing. It will be used in the final document.

Sprint 2 review (Nov 1): Demo: add medicine, receive batch, see stock. Retrospective.

SPRINT 3: POS, Sales and Alerts (Nov 2 to Nov 15)

Goal in simple words: The pharmacy can actually sell things. The system picks the right batch automatically and warns about expiring and low-stock medicines.

Sprint 3 is successful when: you can sell medicines at the POS, stock goes down from the oldest batch first, and the dashboard shows alerts.

Week 5 (Nov 2 to Nov 8): Billing Engine and Expiry Logic

Member 1: Daily expiry check

What: A database function plus a daily alarm (node-cron).
How:

1. In SQL, write a function `generate_expiry_alerts()` that finds every batch with `expiry_date <= today + 7 months` and `quantity > 0`, and inserts an alert (skip it if the same alert already exists, so there are no repeats).
2. In `src/jobs/expiryJob.js` use `cron.schedule('0 6 * * *', ...)` to run it every day at 6:00 AM, which calls `SELECT generate_expiry_alerts();`.
3. Also run it once when the server starts, in case the computer was off at 6 AM.
4. Add `console.log` lines so you can see it ran.

Member 2: Dashboard

How:

1. Build the cards: Today's Sales, Low Stock count, Expiring Soon count.
2. For now show placeholder numbers; connect to the live API next week.
3. Add a simple chart (package `fl_chart`) of sales for the last 7 days.

Member 3: Alerts API

How:

1. `GET /api/alerts` returns alerts, newest first, with optional `?type=expiry` and `?unread=true`.
2. `PUT /api/alerts/:id/read` marks one as read.
3. `GET /api/alerts/summary` returns counts (used by the dashboard cards).

Member 4: CRITICAL TASK. Sales and Billing API with FIFO

What: `POST /api/sales`
FIFO explained simply: *First In, First Out.* Imagine milk in a fridge. You always take the one that expires first. Our system does the same with batches.

How (step by step):

1. Start a **database transaction** (`BEGIN`). A transaction means "all or nothing": if anything fails, everything is undone, so we never get half a sale.
2. For each item in the cart, get its batches that still have stock, **not expired**, sorted by `expiry_date ASC`:

```sql
SELECT id, quantity FROM batches
WHERE medicine_id = $1 AND quantity > 0 AND expiry_date > CURRENT_DATE
ORDER BY expiry_date ASC
FOR UPDATE;   -- locks these rows so two cashiers cannot take the same stock
```

3. Take stock from the first batch. If it is not enough, take what is left and move to the next batch. Example: need 10; batch A has 4, batch B has 20, so take 4 from A and 6 from B.
4. If the total stock across all batches is not enough, `ROLLBACK` and reply "Not enough stock."
5. Save one row in `sales`, and one row in `sale_items` for **each batch used** (so we know exactly where each item came from).
6. If payment is credit, add the total to the customer's `credit_balance`.
7. `COMMIT` and send back the bill details.

Done when: selling 5 items lowers exactly the right batch by 5, and a failed sale changes nothing.

Member 5: Birthday notifications

How:

1. Create a daily node-cron job (for example 8:00 AM) that finds customers whose birthday (month and day) is today.
2. Send a message using `firebase-admin` (FCM) or an SMS service. Keep this in a separate file so the sending method can be swapped (SMS gateways usually need a paid account; FCM is free).
3. Log every message sent so none are sent twice.
4. Write a test endpoint `POST /api/test/birthday` (for development only) to try it quickly.

> **Note:** FCM sends to apps that have registered a device token, so customers must have the app or the pharmacy must use SMS. Decide this early with the pharmacy owner.

Member 6: Test the Sales API

How:

1. Pick a medicine with 2 batches. Write down their quantities.
2. Sell 5. Check that the **oldest-expiry** batch decreased by exactly 5 (or emptied first and the rest came from the next batch).
3. Test the bad cases: sell more than exists, sell an expired batch, send an empty cart, send a negative quantity.

4. Test two sales at the same time (use Postman Runner) to check the stock is still correct.
5. Record every result in the test case table.

Week 6 (Nov 9 to Nov 15): Integration and Polish

Member 1: Low-stock alert after a sale

How:

1. Create a PostgreSQL **trigger** on `sale_items` (or call a function at the end of the sale transaction). After each sale, compute the medicine's total stock; if it is at or below `reorder_level`, insert a `low_stock` alert (if one does not already exist).
2. Test with a medicine that has stock 6 and reorder level 5, then sell 2.

Member 2: Live dashboard

How:

1. Replace placeholder numbers with real calls: `/api/alerts/summary` and a new `/api/reports/today-sales` (agree who builds it with Member 6).
2. Add pull-to-refresh and automatic refresh every minute.
3. Tapping a card opens the matching list.

Member 3: Alerts screen

How:

1. List alerts with icons (clock = expiry, box = low stock). Unread ones in bold.
2. Tap or swipe to mark as read, calling `PUT /api/alerts/:id/read`.
3. Add filter tabs: All, Expiry, Low Stock.

Member 4: Finish the POS

How:

1. Search medicines (use a small delay, called "debounce," so it does not search on every key press).
2. Add to cart, change quantity with + and −, remove items.
3. Calculate the total (and discount if the pharmacy uses it). Show stock warnings if the quantity is more than available.
4. Pay button calls `POST /api/sales`, shows success or error, then clears the cart.

Member 5: Credit sales on the POS

How:

1. In the POS payment step, add the choice **Cash / Credit**.

2. If Credit, show a customer picker (search from the directory) and require a customer.
3. After the sale, the customer's balance updates. Show their new balance on the receipt screen.
4. Optionally warn if the customer already owes a lot.

Member 6: Documentation

How:

1. Write **Phase 4 (Development)**: tools used, how the code is organised, key algorithms (FIFO, cron jobs), screenshots.
2. Write **Phase 5 (Testing)**: the test plan and the results table from the test cases collected so far.
3. Run a full test: add medicine, receive batch, sell it, check alerts.

Sprint 3 review (Nov 15): Demo: full sale with FIFO, an expiry alert, and a low-stock alert. Retrospective. **Prepare for code freeze.**

SPRINT 4: Reporting, Polish and Handover (Nov 16 to Nov 29)

Goal in simple words: Add reports, fix bugs, test with real pharmacy people, install at the pharmacy, and present to the university.

Sprint 4 is successful when: the pharmacy runs the app on its own computer, staff know how to use it, and the documents and presentation are finished.

Week 7 (Nov 16 to Nov 22): Reporting and Bug Squashing

Member 1: Code freeze

What: From now on, **no new features**. Only fixes.
How:

1. Review all remaining PRs.
2. Add database indexes for slow queries (check with `EXPLAIN ANALYZE`).
3. Remove debug logs; make sure secrets are only in `.env`.
4. Add basic protection: `helmet` and rate limiting on the login route.
5. Set up an automatic daily database backup script (`pg_dump`) for the pharmacy.

Member 2: Responsive design

How:

1. Get the pharmacy's actual screen size (ask them, or take a photo or measure at the pharmacy). Test the app at that exact size.

2. Use `LayoutBuilder` or `MediaQuery` so desktop shows tables and mobile shows lists.
3. Test on a real Android phone and in the Windows app. Fix overflow errors (yellow-black stripes).

Member 3: Supplier Return screen

How:

1. Create `GET /api/batches/expiring?months=7` (batches with stock that expire in the next 7 months, grouped by supplier).
2. Build a screen with a table: medicine, batch, supplier, expiry date, quantity left, with a filter by supplier.
3. Optional: a "Mark as returned" button that sets the batch quantity to 0 and records a note.

Member 4: PDF receipts

How:

1. Add the Flutter packages `pdf` and `printing`.
2. Build the receipt layout: pharmacy name and address, bill number, date, item table (name, qty, price), total, thank-you line.
3. Add buttons: Print, Save as PDF, Share.
4. Test on a small thermal-printer size if the pharmacy uses one (usually 58mm or 80mm), and also on A4.

Member 5: Outstanding Credit Report

How:

1. Screen showing each customer with unpaid credit: name, phone, amount owed, last payment date, with a total at the bottom.
2. Add a "Record payment" button on each row.
3. Add sorting (highest debt first) and search.

Member 6: Exportable reports and first UAT

How:

1. Backend: `GET /api/reports/sales?from=&to=` and `GET /api/reports/stock`.
2. Export to CSV (simple) and Excel (`exceljs` on the server or the `excel` package in Flutter). Include totals.
3. Add Export buttons to the screens.
4. Run **internal UAT**: the whole team uses the app like real staff, following a script (login, add medicine, receive stock, sell, credit sale, reports). Write down every problem and give

each to the right member.

Week 8 (Nov 23 to Nov 29): Deployment and Final Presentation

Member 1: Install at the pharmacy

How:

1. Install Node.js and PostgreSQL on the pharmacy's desktop. Restore the database from the SQL scripts.
2. Copy the backend; create the production `.env` with a **new strong** `JWT_SECRET` and a strong database password.
3. Make the server start automatically when Windows starts (use `pm2` with `pm2-windows-startup`, or a Windows Service).
4. Allow the port in Windows Firewall so the Android phone on the same Wi-Fi can reach it (example: `http://192.168.1.10:3000`). Give the desktop a fixed local IP address.
5. Set up the automatic backup job and test restoring a backup.

Member 2: Build the final apps

How:

1. Set the final server address in the app's settings or config.
2. Windows: `flutter build windows --release`, then package it (Inno Setup or MSIX) as an installer `.exe`.
3. Android: `flutter build apk --release`.
4. Give the app a proper name and icon.
5. Test both builds on clean machines.

Members 3 and 4: Move the old data

How:

1. Ask the pharmacy for their existing records (notebooks, Excel sheets).
2. Make a clean spreadsheet template with the same columns as the database (medicine, category, supplier, batch, quantity, expiry, price).
3. Write a small import script (read the CSV and insert into the tables) or use pgAdmin's import tool.
4. Check the data after import: counts match, no blank expiry dates.
5. Keep a copy of the original data in case of mistakes.

Member 5: Train the staff

How:

1. Prepare a short **user guide** with screenshots (login, add customer, make a sale, credit sale, receipt).
2. Run a hands-on session: let staff do the tasks themselves.
3. Give them a one-page "cheat sheet."
4. Collect feedback and note any final requests.

Member 6: Finish documents and presentation

How:

1. Finalize the SRS, System Design, Development, Testing, and User Guide. Add the final test results, screenshots, and a conclusion with future improvements.
2. Collect **UAT sign-off** (the pharmacy owner confirms in writing that the system works).
3. Build the defense PowerPoint: problem, solution, architecture, demo screenshots or video, testing results, challenges, future work.
4. Hold a practice defense with the whole team. Prepare answers to likely questions (Why Flutter? Why PostgreSQL? How does FIFO work? How is data secured?).

Final review and handover (Nov 29): pharmacy is live, documents submitted, presentation ready.

---

7. Timeline at a Glance

Week	Dates	Main Result
1	Oct 5 to 11	Tools ready, designs drawn, database created
2	Oct 12 to 18	Login works, customer APIs, POS shell
3	Oct 19 to 25	Medicine add/edit works
4	Oct 26 to Nov 1	Batches and suppliers work
5	Nov 2 to 8	Sales with FIFO, expiry job, alerts API
6	Nov 9 to 15	Full POS, dashboard, alerts and credit sales
7	Nov 16 to 22	Reports, PDFs, bug fixes (code freeze)
8	Nov 23 to 29	Install, train, defend, hand over

---

8. What Does "Done" Mean? (Final Checklist)

- [ ] Staff can log in securely
- [ ] Medicines, categories, and suppliers can be added and edited
- [ ] Batches with expiry dates can be received
- [ ] A sale takes stock from the soonest-expiring batch first
- [ ] Expiry alerts (7 months) appear every day automatically
- [ ] Low-stock alerts appear after sales
- [ ] Credit sales work and the Outstanding Credit Report is correct
- [ ] Birthday notifications are sent
- [ ] PDF receipts can be printed or saved
- [ ] Sales and stock reports export to Excel/CSV
- [ ] Supplier Return screen works
- [ ] Windows installer and Android APK are built
- [ ] System runs on the pharmacy computer and starts by itself
- [ ] Old data is imported and checked
- [ ] Staff are trained and UAT is signed off
- [ ] All documentation and defense slides are finished

---

9. Risks and How to Handle Them

What could go wrong	How to prevent or fix it
A teammate is blocked waiting for another's API	Agree on the JSON format first (Week 1), and use fake data in the app until the real API is ready
FIFO logic has bugs (the most important feature)	Member 4 starts early in Week 5, Member 6 tests heavily, and the transaction and row lock prevent double-selling
Merge conflicts in GitHub	Small branches, pull `develop` every morning, and merge often
Pharmacy computer is slow or old	Keep queries simple, use indexes, and test early on similar hardware
Power cut corrupts data	Daily automatic backups plus restore practice
SMS gateway costs money	Start with FCM (free); only add SMS if the owner agrees to pay
Running out of time	The Critical Path is: login, medicines, batches, sales with FIFO, alerts. Reports and polish are cut first if needed
Pharmacy staff find the app confusing	Show Figma designs early and run training and UAT
A member is sick or unavailable	Pairs for the key tasks: Members 3 and 4 share stock knowledge; Members 1 and 6 share backend knowledge

---

10. Quick Glossary

- **Backend:** the hidden brain on the server.
- **Frontend:** the screens people see.
- **API / Endpoint:** a doorway on the server that the app calls.
- **Database / Table:** the storage and its spreadsheets.
- **Batch:** one delivery of a medicine, with its own expiry date.
- **FIFO:** First In, First Out. Sell the oldest (soonest to expire) first.
- **POS:** Point of Sale, the checkout screen.
- **Transaction:** a group of database steps that all succeed together or all get undone.
- **Trigger:** an automatic database action after something happens.
- **JWT:** a digital wristband proving you logged in.
- **Cron job:** a task that runs on a schedule, like an alarm clock.

- **Sprint:** a 2-week work cycle.
- **UAT:** the real users test and approve the system.
- **Code freeze:** no new features, only fixes.